

LENGUAJES DE PROGRAMACIÓN I

Fundamentos, Algoritmos y Aplicaciones Prácticas

Autor:

Yuber Anthony Ticona Incacutipa

Unidad 2

LENGUAJES DE PROGRAMACIÓN I

Fundamentos, Algoritmos y Aplicaciones Prácticas

Primera edición, 2026

© 2026 Yuber Anthony Ticona Incacutipa

Todos los derechos reservados. Se permite la reproducción parcial con fines educativos citando la fuente.

El autor ha hecho todos los esfuerzos razonables para verificar que la información contenida en este libro es correcta en el momento de su publicación. Sin embargo, ni el autor ni el editor asumen responsabilidad por errores u omisiones, o por daños resultantes del uso de la información aquí contenida.

Todos los ejemplos de código presentados en este libro son de carácter educativo y han sido verificados por el autor.

Impreso y distribuido con fines académicos

PRÓLOGO

Este libro nació de la necesidad de contar con un material de estudio claro, completo y actualizado para los estudiantes que se inician en el fascinante mundo de la programación. En una época donde los lenguajes de programación evolucionan constantemente y las demandas del mercado tecnológico crecen sin cesar, dominar los fundamentos sigue siendo la clave para enfrentar cualquier desafío computacional.

A lo largo de estas páginas encontrarás una guía cuidadosamente estructurada que comienza con los fundamentos del análisis de complejidad computacional y avanza progresivamente hacia temas más especializados como el manejo de archivos, las estructuras de datos, la programación en múltiples lenguajes modernos y el desarrollo de interfaces gráficas.

Los ejemplos de código que encontrarás aquí han sido diseñados para ser didácticos, funcionales y reutilizables. Cada capítulo incluye recuadros de definición, notas de buenas prácticas, tablas comparativas y ejercicios prácticos que refuerzan el aprendizaje.

Mi deseo es que este libro se convierta en un compañero de estudio confiable, que te acompañe no solo durante tu formación académica sino también en los primeros años de tu vida profesional.

Yuber Anthony Ticona Incacutipa
2026

ÍNDICE GENERAL

Prólogo	ii
Capítulo 1. Análisis de Complejidad Computacional	1
1.1 Notación Big-O	1
1.2 Complejidad Temporal y Espacial	3
1.3 Comparativa de Algoritmos	5
1.4 Estructuras de Datos y sus Complejidades	7
Capítulo 2. Manejo de Archivos en C	10
2.1 Escritura Básica de Texto	10
2.2 Lectura Línea por Línea	12
2.3 Archivos Binarios con fwrite/fread	14
Capítulo 3. Manejo de Múltiples Archivos en C	17
3.1 Escribir y Leer Múltiples Archivos	17
3.2 Filtrar, Copiar y Consolidar	19
3.3 Patrones y Comandos Clave	22
Capítulo 4. Estructuras (struct) en C	24
4.1 ¿Qué es una Estructura?	24
4.2 Sintaxis Básica y typedef	25
4.3 Arreglos de Estructuras	27
4.4 Estructuras con Funciones y Archivos	29
Capítulo 5. Otros Lenguajes de Programación	32
5.1 PHP	32
5.2 Java	34
5.3 JavaScript / Node.js	36
5.4 Go	38
5.5 Rust	40
5.6 Mojo	42
5.7 Tabla Comparativa	44
Capítulo 6. Interfaces Gráficas (GUI)	46
6.1 GUIs en C con GTK4	46
6.2 Frameworks por Lenguaje	48
6.3 Ejemplo con Python Tkinter	50

Capítulo 7. Práctica Integradora Final	53
7.1 Sistema de Inventario en C	53
7.2 Benchmark de Búsquedas	56
7.3 Ejercicios de Completar Código	58
7.4 Proyecto Final Integrador	60
 Bibliografía	 63

CAPÍTULO 1

Análisis de Complejidad Computacional

El análisis de complejidad computacional constituye uno de los pilares fundamentales de la ciencia de la computación. Permite al programador predecir, antes de ejecutar un programa, cuánto tiempo y cuánta memoria requerirá a medida que el tamaño de la entrada crece. Esta capacidad analítica es esencial para diseñar sistemas eficientes, escalables y competitivos.

En la era actual, donde los volúmenes de datos crecen exponencialmente y los sistemas de tiempo real exigen respuestas en milisegundos, la diferencia entre un algoritmo $O(n)$ y uno $O(n^2)$ puede ser la diferencia entre una aplicación exitosa y un sistema que colapsa bajo carga.

1.1 Notación Big-O

La notación Big-O es el lenguaje matemático que usamos para describir el comportamiento asintótico de un algoritmo. Se concentra en cómo crece el tiempo de ejecución (o el uso de memoria) cuando el tamaño de la entrada tiende a infinito, ignorando constantes y términos de menor orden porque son irrelevantes a gran escala.

Definición Formal — Big-O

Se dice que $f(n) = O(g(n))$ si existen constantes positivas c y n_0 tales que $f(n) \leq c \cdot g(n)$ para todo $n \geq n_0$. En la práctica esto significa: $g(n)$ es una cota superior del crecimiento de $f(n)$ a partir de cierto punto.

La tabla siguiente resume las notaciones más frecuentes, ordenadas de menor a mayor complejidad:

Notación	Nombre	Ejemplo clásico	Crece lento?
$O(1)$	Constante	Acceso a array por índice	Sí, ideal
$O(\log n)$	Logarítmica	Búsqueda binaria	Muy bien
$O(n)$	Lineal	Recorrer un array	Bien
$O(n \log n)$	Linealítmica	Merge Sort, Heap Sort	Aceptable
$O(n^2)$	Cuadrática	Bubble Sort, Selection Sort	Malo
$O(2^n)$	Exponencial	Generar todos los subconjuntos	Muy malo
$O(n!)$	Factorial	Permutaciones, TSP fuerza bruta	Catastrófico

Ejemplo práctico — cómo identificar la complejidad de un fragmento de código:

```
// O(1) – tiempo constante: no depende de n
int primerElemento(int arr[]) {
    return arr[0];
}

// O(n) – un solo ciclo que recorre los n elementos
int sumaArray(int arr[], int n) {
    int suma = 0;
    for (int i = 0; i < n; i++)
        suma += arr[i];
    return suma;
}

// O(n²) – ciclo doble anidado, cada uno llega hasta n
void imprimirPares(int arr[], int n) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            printf("%d,%d ", arr[i], arr[j]);
}
```

1.2 Complejidad Temporal y Espacial

Todo algoritmo consume dos recursos esenciales: tiempo de CPU y memoria RAM. Analizamos ambos por separado porque a veces un algoritmo puede ser rápido (baja complejidad temporal) pero consumir mucha memoria (alta complejidad espacial), y viceversa.

La complejidad temporal cuenta las operaciones elementales que realiza el algoritmo. La complejidad espacial mide la memoria auxiliar extra que necesita (sin contar la entrada).

Regla práctica

La complejidad espacial $O(1)$ significa que el algoritmo usa una cantidad fija de variables extra, sin importar el tamaño de la entrada. Es el ideal cuando los datos ya están en memoria.

Ejemplo comparativo — búsqueda lineal vs búsqueda binaria:

```
// — Búsqueda LINEAL — O(n) tiempo, O(1) espacio
int busquedaLineal(int arr[], int n, int objetivo) {
    for (int i = 0; i < n; i++)
        if (arr[i] == objetivo) return i;
    return -1;
}

// — Búsqueda BINARIA — O(log n) tiempo, O(1) espacio
// REQUISITO: el array debe estar ordenado
int busquedaBinaria(int arr[], int n, int objetivo) {
    int izq = 0, der = n - 1;
    while (izq <= der) {
```

```

        int medio = izq + (der - izq) / 2; // evita overflow
        if (arr[medio] == objetivo) return medio;
        else if (arr[medio] < objetivo) izq = medio + 1;
        else der = medio - 1;
    }
    return -1;
}

// Con n = 1,000,000:
// Lineal → hasta 1.000.000 comparaciones (peor caso)
// Binaria → máximo 20 comparaciones (log2 1.000.000 ≈ 20)

```

1.3 Comparativa de Algoritmos de Ordenamiento

El ordenamiento es uno de los problemas más estudiados en computación. Existen docenas de algoritmos, cada uno con sus fortalezas y debilidades. La tabla siguiente resume los más importantes:

Algoritmo	Mejor caso	Caso promedio	Peor caso	Espacio	Estable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sí
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sí
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sí
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Sí
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$	Sí

Implementación de Merge Sort — el algoritmo de ordenamiento más elegante:

```

#include <stdlib.h>

// Fusiona dos subarreglos ordenados en uno —  $O(n)$ 
void fusionar(int arr[], int izq, int medio, int der) {
    int n1 = medio - izq + 1;
    int n2 = der - medio;
    int *L = malloc(n1 * sizeof(int));
    int *R = malloc(n2 * sizeof(int));

    for (int i=0; i<n1; i++) L[i] = arr[izq+i];
    for (int j=0; j<n2; j++) R[j] = arr[medio+1+j];
}

```



```

    int i=0, j=0, k=izq;
    while (i<n1 && j<n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i<n1) arr[k++] = L[i++];
    while (j<n2) arr[k++] = R[j++];

    free(L); free(R);
}

// Merge Sort recursivo - O(n log n) tiempo, O(n) espacio
void mergeSort(int arr[], int izq, int der) {
    if (izq < der) {
        int medio = izq + (der - izq) / 2;
        mergeSort(arr, izq, medio);
        mergeSort(arr, medio+1, der);
        fusionar(arr, izq, medio, der);
    }
}

```

1.4 Estructuras de Datos y sus Complejidades

La elección de la estructura de datos correcta es tan importante como la elección del algoritmo. A continuación, un resumen de las operaciones fundamentales y sus complejidades:

Estructura	Acceso	Búsqueda	Inserción	Eliminación	Espacio
Array	O(1)	O(n)	O(n)	O(n)	O(n)
Lista Enlazada	O(n)	O(n)	O(1)*	O(1)*	O(n)
Pila (Stack)	O(n)	O(n)	O(1)	O(1)	O(n)
Cola (Queue)	O(n)	O(n)	O(1)	O(1)	O(n)
Hash Table	N/A	O(1)**	O(1)**	O(1)**	O(n)
BST (bal.)	O(log n)	O(log n)	O(log n)	O(log n)	O(n)
Heap	O(1)***	O(n)	O(log n)	O(log n)	O(n)

(*) Si tenemos la referencia directa al nodo. (**) Caso promedio, asumiendo buena función hash. (***) Solo el máximo/mínimo.

Resumen del Capítulo 1

La notación Big-O nos da un lenguaje común para comparar algoritmos independientemente del hardware. Los algoritmos $O(n \log n)$ como Merge Sort son la mejor opción general para ordenamiento. La búsqueda binaria ($O(\log n)$) supera enormemente a la lineal ($O(n)$) en datos ordenados. Siempre elegir la estructura de datos adecuada al problema.

CAPÍTULO 2

Manejo de Archivos en C

La persistencia de datos es una necesidad fundamental en cualquier aplicación real. Los programas necesitan guardar información entre ejecuciones: configuraciones, registros de usuarios, resultados de cálculos, logs de operaciones. En C, el manejo de archivos se realiza a través de la biblioteca estándar `<stdio.h>`, que provee una interfaz portátil y completa para trabajar con archivos tanto de texto como binarios.

Ciclo de vida de un archivo en C

Todo manejo de archivo sigue el mismo patrón: (1) Abrir con `fopen()`, verificando que no retorne `NULL`. (2) Operar: leer, escribir, mover el cursor. (3) Cerrar con `fclose()` para liberar el descriptor y vaciar los buffers. Olvidar el paso 3 puede corromper archivos o provocar pérdida de datos.

2.1 Escritura Básica de Texto

Para escribir datos en un archivo de texto se usa `fopen()` con el modo `"w"` (crear/sobreescribir) o `"a"` (agregar al final). Las funciones de escritura más usadas son `fprintf()` para escritura con formato y `fputs()` para cadenas simples.

Modo	Comportamiento	Archivo inexistente	Puntero inicial
"r"	Solo lectura	Error (NULL)	Inicio
"w"	Solo escritura	Lo crea	Inicio (borra)
"a"	Solo escritura al final	Lo crea	Final
"r+"	Lectura y escritura	Error (NULL)	Inicio
"w+"	Lectura y escritura	Lo crea	Inicio (borra)
"a+"	Lectura y escritura	Lo crea	Final

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    /* — Paso 1: Abrir el archivo — */
    FILE *archivo = fopen("registro.txt", "w");
    if (archivo == NULL) {
        perror("Error al crear registro.txt");
        return EXIT_FAILURE;
    }
}
```

```

/* — Paso 2: Escribir datos con formato — */
fprintf(archivo, "=== Registro de Estudiantes ===\n");
fprintf(archivo, "%-20s %5s %8s\n", "Nombre", "Edad", "Promedio");
fprintf(archivo, "%-20s %5d %8.2f\n", "Ana Garcia", 20, 9.5);
fprintf(archivo, "%-20s %5d %8.2f\n", "Luis Martinez", 22, 8.3);
fprintf(archivo, "%-20s %5d %8.2f\n", "Sofia Ruiz", 21, 9.8);

/* — Paso 3: Cerrar siempre — */
fclose(archivo);
printf("Archivo creado exitosamente.\n");
return EXIT_SUCCESS;
}

```

2.2 Lectura Línea por Línea

La función `fgets()` es la herramienta más segura para leer líneas de texto. Recibe un buffer, su tamaño máximo y el puntero al archivo. Lee hasta encontrar `'\n'` o EOF, y siempre agrega el carácter nulo `'\0'` al final, garantizando que la cadena esté bien formada.

```

#include <stdio.h>
#include <string.h>

int main() {
    FILE *archivo = fopen("registro.txt", "r");
    if (archivo == NULL) {
        perror("No se pudo abrir registro.txt");
        return 1;
    }

    char linea[256];
    int numLinea = 0;
    int totalLineas = 0;

    printf("Contenido del archivo:\n");
    printf("%-4s %s\n", "#", "Línea");
    printf("%s\n", "-----");

    while (fgets(linea, sizeof(linea), archivo) != NULL) {
        /* Eliminar el '\n' del final si existe */
        size_t len = strlen(linea);
        if (len > 0 && linea[len-1] == '\n')
            linea[len-1] = '\0';

        /* Saltar líneas vacías */
        if (strlen(linea) == 0) { totalLineas++; continue; }

        printf("%-4d %s\n", ++numLinea, linea);
        totalLineas++;
    }
}

```

```

printf("\nTotal líneas procesadas: %d\n", totalLineas);
fclose(archivo);
return 0;
}

```

2.3 Archivos Binarios con fwrite/fread

Los archivos binarios guardan los bytes exactos de las variables en memoria, sin conversión a texto. Son más eficientes en tamaño y velocidad que los archivos de texto, y son ideales para guardar estructuras de datos directamente.

fwrite vs fprintf

`fwrite(ptr, tamaño, cantidad, archivo)` escribe 'cantidad' bloques de 'tamaño' bytes cada uno. Para un struct de 100 bytes, `fwrite` escribe exactamente 100 bytes. `fprintf` escribiría una representación textual que podría ocupar más espacio y requeriría parseo al leer.

```

#include <stdio.h>
#include <string.h>

typedef struct {
    int id;
    char nombre[50];
    int edad;
    float promedio;
} Estudiante;

/* — GUARDAR — O(n) */
int guardarEstudiantes(const char *ruta,
                      Estudiante *arr, int n) {
    FILE *f = fopen(ruta, "wb");
    if (!f) { perror(ruta); return -1; }

    size_t ok = fwrite(arr, sizeof(Estudiante), n, f);
    fclose(f);
    printf("%zu estudiantes guardados en '%s'\n", ok, ruta);
    return (int)ok;
}

/* — CARGAR — O(n) */
int cargarEstudiantes(const char *ruta,
                     Estudiante *arr, int maxN) {
    FILE *f = fopen(ruta, "rb");
    if (!f) { perror(ruta); return -1; }

    int leidos = (int)fread(arr, sizeof(Estudiante), maxN, f);
    fclose(f);
    return leidos;
}

```

```
int main() {
    Estudiante bd[] = {
        {1, "Ana Garcia",    20, 9.5f},
        {2, "Luis Martinez", 22, 8.3f},
        {3, "Sofia Ruiz",    21, 9.8f},
    };
    int n = sizeof(bd)/sizeof(bd[0]);

    guardarEstudiantes("estudiantes.dat", bd, n);

    Estudiante cargados[100];
    int total = cargarEstudiantes("estudiantes.dat",
                                cargados, 100);
    printf("\nRegistros leídos: %d\n", total);
    for (int i = 0; i < total; i++)
        printf("[%d] %-20s edad:%2d prom:%.1f\n",
               cargados[i].id, cargados[i].nombre,
               cargados[i].edad, cargados[i].promedio);
    return 0;
}
```

Resumen del Capítulo 2

El manejo de archivos en C sigue siempre el patrón: abrir → operar → cerrar. `fgets()` es la función más segura para leer texto línea a línea. Los archivos binarios con `fwrite/fread` son más eficientes para estructuras de datos. Siempre verificar que `fopen()` no retorne NULL y siempre llamar `fclose()`.

CAPÍTULO 3

Manejo de Múltiples Archivos en C

En aplicaciones reales, los datos raramente se concentran en un único archivo. Es común trabajar con múltiples fuentes: datos de diferentes departamentos, registros por fecha, archivos de configuración y de datos. Este capítulo aborda los patrones y técnicas para gestionar eficientemente múltiples archivos en C.

3.1 Escribir y Leer Múltiples Archivos

```
#include <stdio.h>
#include <stdlib.h>

#define NUM_ARCHIVOS 4

int main() {
    /* Nombres y contenidos de los archivos */
    const char *nombres[NUM_ARCHIVOS] = {
        "dept_sistemas.txt", "dept_ventas.txt",
        "dept_rrhh.txt",      "dept_contab.txt"
    };
    const char *departamentos[NUM_ARCHIVOS] = {
        "Sistemas", "Ventas", "RRHH", "Contabilidad"
    };
    int empleados[NUM_ARCHIVOS] = { 12, 25, 8, 15 };

    /* — ESCRIBIR: generar un archivo por departamento — */
    for (int i = 0; i < NUM_ARCHIVOS; i++) {
        FILE *f = fopen(nombres[i], "w");
        if (!f) { perror(nombres[i]); continue; }

        fprintf(f, "Departamento: %s\n", departamentos[i]);
        fprintf(f, "Empleados:      %d\n", empleados[i]);
        fprintf(f, "Estado:          Activo\n");
        fclose(f);
        printf("Creado: %s\n", nombres[i]);
    }

    /* — LEER: mostrar todos los archivos — */
    printf("\n=== Contenido de todos los archivos ===\n");
    char linea[256];
    for (int i = 0; i < NUM_ARCHIVOS; i++) {
        FILE *f = fopen(nombres[i], "r");
        if (!f) { perror(nombres[i]); continue; }
        printf("\n--- %s ---\n", nombres[i]);
        while (fgets(linea, sizeof(linea), f))
            printf("  %s", linea);
        fclose(f);
    }
    return 0;
}
```

3.2 Filtrar, Copiar y Consolidar Archivos

Un patrón fundamental en procesamiento de datos: leer múltiples archivos de entrada, aplicar un filtro, y consolidar los resultados que cumplen el criterio en un único archivo de salida.

```
#include <stdio.h>
#include <string.h>

/* Filtra líneas que contienen 'filtro' y las escribe en fSalida */
int procesarArchivo(FILE *fSalida, const char *entrada,
                    const char *filtro) {
    FILE *f = fopen(entrada, "r");
    if (!f) { perror(entrada); return 0; }

    char linea[512];
    int coincidencias = 0;

    fprintf(fSalida, "\n=== Fuente: %s ===\n", entrada);
    while (fgets(linea, sizeof(linea), f)) {
        if (strstr(linea, filtro) != NULL) {
            fputs(linea, fSalida);
            coincidencias++;
        }
    }
    fprintf(fSalida, "    [%d coincidencias]\n", coincidencias);
    fclose(f);
    return coincidencias;
}

int main() {
    const char *archivos[] = {
        "dept_sistemas.txt", "dept_ventas.txt",
        "dept_rrhh.txt",    "dept_contab.txt"
    };
    int n = 4;
    const char *filtro = "Activo";
    const char *salida = "consolidado.txt";

    FILE *fOut = fopen(salida, "w");
    if (!fOut) { perror(salida); return 1; }

    fprintf(fOut, "REPORTE CONSOLIDADO - Filtro: '%s'\n", filtro);
    fprintf(fOut, "=====\n");

    int total = 0;
    for (int i = 0; i < n; i++)
        total += procesarArchivo(fOut, archivos[i], filtro);

    fprintf(fOut, "\n=====");
    fprintf(fOut, "TOTAL DE COINCIDENCIAS: %d\n", total);
    fclose(fOut);
    printf("Consolidado generado: %s (%d coincidencias)\n",

```

```

        salida, total);
    return 0;
}

```

3.3 Patrones y Comandos Clave

La siguiente tabla resume las funciones esenciales para el manejo de archivos en C, organizadas por categoría:

Categoría	Función	Descripción
Apertura/Cierre	fopen(ruta, modo)	Abre archivo, retorna FILE* o NULL
Apertura/Cierre	fclose(fp)	Cierra y libera el descriptor
Escritura texto	fprintf(fp, fmt, ...)	Escribe con formato printf
Escritura texto	fputs(str, fp)	Escribe cadena de caracteres
Lectura texto	fgets(buf, n, fp)	Lee línea de forma segura
Lectura texto	fscanf(fp, fmt, ...)	Lee con formato scanf
Binario	fwrite(ptr, sz, n, fp)	Escribe n bloques de sz bytes
Binario	fread(ptr, sz, n, fp)	Lee n bloques de sz bytes
Posición	fseek(fp, offset, from)	Mueve el cursor del archivo
Posición	ftell(fp)	Retorna la posición actual
Posición	rewind(fp)	Regresa al inicio del archivo
Estado	feof(fp)	Verdadero si se llegó al final
Estado	ferror(fp)	Verdadero si ocurrió un error
Sistema	remove(ruta)	Elimina un archivo del disco
Sistema	rename(viejo, nuevo)	Renombra o mueve un archivo

Resumen del Capítulo 3

El patrón de múltiples archivos se basa en iterar sobre una lista de nombres, abrir/procesar/cerrar cada uno. La función strstr() permite filtrar líneas por contenido. Consolidar resultados en un archivo de salida es un patrón muy común en sistemas de reportes y ETL.

CAPÍTULO 4

Estructuras (struct) en C

Las estructuras son la herramienta de C para crear tipos de datos compuestos: agrupan múltiples variables de tipos posiblemente distintos bajo un único identificador. Representan el primer paso hacia la programación orientada a objetos y son indispensables para modelar entidades del mundo real directamente en el código.

4.1 ¿Qué es una Estructura?

Una estructura (struct) es una colección de campos (variables miembro) que pueden ser de cualquier tipo: enteros, flotantes, cadenas, arreglos, punteros, o incluso otras estructuras. A diferencia de un array, los campos tienen nombres descriptivos y pueden tener tipos distintos.

Analogía con el mundo real

Imagina la ficha de un empleado: tiene número de empleado (entero), nombre (cadena), departamento (cadena), salario (decimal) y fecha de ingreso (estructura de fecha). En C eso es exactamente un struct Empleado. El struct nos permite manipular todos esos datos como una unidad coherente.

4.2 Sintaxis Básica y typedef

```
/* — Forma clásica (requiere escribir 'struct' al declarar) — */
struct Punto {
    float x;
    float y;
};
struct Punto p1;

/* — Forma moderna con typedef (RECOMENDADA) — */
typedef struct {
    float x;
    float y;
} Punto;
Punto p1;    /* Sin 'struct', más limpio */

/* — Inicialización — */
Punto p2 = {3.5f, 7.2f};          /* por posición */
Punto p3 = { .x = 1.0f, .y = 2.0f }; /* por nombre (C99+) */

/* — Acceso a campos: operador punto (.) — */
p1.x = 10.0f;
p1.y = 20.0f;
printf("Punto: (%.1f, %.1f)\n", p1.x, p1.y);
```

```

/* — Acceso via puntero: operador flecha (->) — */
Punto *ptr = &p1;
printf("Via puntero: (%.1f, %.1f)\n", ptr->x, ptr->y);
/* ptr->x equivale exactamente a (*ptr).x */

/* — Asignación de estructuras (copia completa) — */
Punto p4 = p1; /* p4 es una copia independiente de p1 */

```

4.3 Arreglos de Estructuras

Los arreglos de estructuras son el patrón más común en C para manejar colecciones de entidades. Representan bases de datos en memoria: listas de estudiantes, empleados, productos, etc.

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

typedef struct {
    int id;
    char nombre[60];
    char departamento[30];
    float salario;
    int anios;
} Empleado;

#define MAX 200

void imprimirEmpleado(const Empleado *e) {
    printf("[%03d] %-30s %-15s $%9.2f %d años\n",
        e->id, e->nombre, e->departamento,
        e->salario, e->anios);
}

/* Ordenar por salario descendente - Insertion Sort O(n^2) */
void ordenarPorSalario(Empleado arr[], int n) {
    for (int i = 1; i < n; i++) {
        Empleado tmp = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j].salario < tmp.salario) {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = tmp;
    }
}

/* Buscar por ID - O(n) */
Empleado* buscarPorId(Empleado arr[], int n, int id) {
    for (int i = 0; i < n; i++)
        if (arr[i].id == id) return &arr[i];
    return NULL;
}

```

```

int main() {
    Empleado empresa[MAX];
    int total = 0;

    /* Agregar empleados */
    empresa[total++] = (Empleado){1,"Elena Torres", "Sistemas", 55000,5};
    empresa[total++] = (Empleado){2,"Marco Rios", "Ventas", 42000,3};
    empresa[total++] = (Empleado){3,"Diana Lopez", "RRHH", 48000,7};
    empresa[total++] = (Empleado){4,"Carlos Perez", "Sistemas", 60000,9};

    printf("=== Lista original ===\n");
    for (int i = 0; i < total; i++) imprimirEmpleado(&empresa[i]);

    ordenarPorSalario(empresa, total);
    printf("\n=== Ordenados por salario ===\n");
    for (int i = 0; i < total; i++) imprimirEmpleado(&empresa[i]);

    Empleado *e = buscarPorId(empresa, total, 3);
    if (e) printf("\nEncontrado ID 3: %s\n", e->nombre);
    return 0;
}

```

4.4 Estructuras con Funciones y Archivos

La combinación de struct + archivos binarios + funciones especializadas es el patrón más poderoso de C para construir sistemas de gestión de datos. Permite acceso directo ($O(1)$) a cualquier registro si conocemos su posición en el archivo.

```

#include <stdio.h>

typedef struct {
    int id;
    char nombre[50];
    float promedio;
    int activo; /* 1=activo, 0=baja */
} Estudiante;

#define ARCHIVO "base_estudiantes.dat"

/* Agregar al final —  $O(1)$  amortizado */
int agregarEstudiante(const Estudiante *e) {
    FILE *f = fopen(ARCHIVO, "ab");
    if (!f) return -1;
    fwrite(e, sizeof(Estudiante), 1, f);
    fclose(f);
    return 0;
}

/* Acceso directo por posición —  $O(1)$  con fseek */
int leerPosicion(int pos, Estudiante *dest) {

```

```

    FILE *f = fopen(ARCHIVO, "rb");
    if (!f) return -1;
    fseek(f, pos * sizeof(Estudiante), SEEK_SET);
    int ok = (int)fread(dest, sizeof(Estudiante), 1, f);
    fclose(f);
    return ok;
}

/* Actualizar registro en posición - O(1) */
int actualizarPosicion(int pos, const Estudiante *e) {
    FILE *f = fopen(ARCHIVO, "r+b");
    if (!f) return -1;
    fseek(f, pos * sizeof(Estudiante), SEEK_SET);
    fwrite(e, sizeof(Estudiante), 1, f);
    fclose(f);
    return 0;
}

/* Contar registros totales - O(1) con fseek al final */
long contarRegistros() {
    FILE *f = fopen(ARCHIVO, "rb");
    if (!f) return 0;
    fseek(f, 0, SEEK_END);
    long total = ftell(f) / sizeof(Estudiante);
    fclose(f);
    return total;
}

```

Resumen del Capítulo 4

Las estructuras permiten modelar entidades del mundo real agrupando campos de distintos tipos. Con `typedef struct` creamos tipos propios reutilizables. Los arreglos de estructuras son bases de datos en memoria. Combinando `struct` con `fwrite/fread` y `fseek` obtenemos acceso directo $O(1)$ a registros en archivos binarios.

CAPÍTULO 5

Otros Lenguajes de Programación

El ecosistema de lenguajes de programación en 2026 es más diverso y dinámico que nunca. Cada lenguaje ha evolucionado para resolver problemas específicos con mayor eficacia. Conocer sus diferencias permite al programador elegir la herramienta correcta para cada tarea.

5.1 PHP

PHP (Hypertext Preprocessor) es el lenguaje de scripting más utilizado en el desarrollo web del lado del servidor. PHP 8.3 introdujo mejoras significativas en el sistema de tipos, rendimiento del JIT y nuevas características sintácticas que modernizan el lenguaje.

```
<?php
// PHP 8.3 – Clases readonly y tipos de intersección

readonly class Estudiante {
    public function __construct(
        public int    $id,
        public string $nombre,
        public float  $promedio,
    ) {}
}

$estudiantes = [
    new Estudiante(1, 'Ana García',    9.5),
    new Estudiante(2, 'Luis Martínez', 8.3),
    new Estudiante(3, 'Sofía Ruiz',    9.8),
];

// Guardar en JSON
file_put_contents(
    'estudiantes.json',
    json_encode($estudiantes, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE)
);

// Leer y filtrar – promedio > 9.0
$datos = json_decode(file_get_contents('estudiantes.json'));
$destacados = array_filter($datos, fn($e) => $e->promedio > 9.0);

foreach ($destacados as $e) {
    echo "{$e->nombre}: {$e->promedio}\n";
}
?>
```

5.2 Java

Java 21 LTS (2023) introdujo características revolucionarias: Records para datos inmutables, Pattern Matching para instanceof, Sealed Classes y Virtual Threads (Project Loom) que permiten millones de hilos ligeros sin la complejidad tradicional de la concurrencia.

```
// Java 21 LTS – Record, Stream API, NIO2
import java.nio.file.*;
import java.util.*;
import java.util.stream.*;

// Record: clase inmutable con equals/hashCode/toString automáticos
record Estudiante(int id, String nombre, double promedio) {}

public class Ejemplo {
    public static void main(String[] args) throws Exception {

        var estudiantes = List.of(
            new Estudiante(1, "Ana García", 9.5),
            new Estudiante(2, "Luis Martínez", 8.3),
            new Estudiante(3, "Sofía Ruiz", 9.8)
        );

        // Serializar a JSON simple (sin dependencias)
        var json = estudiantes.stream()
            .map(e -> String.format(
                "{\"id\":%d,\"nombre\":\"%s\",\"promedio\":%.1f}",
                e.id(), e.nombre(), e.promedio()))
            .collect(Collectors.joining(",\n", "[\n", "\n]"));

        Files.writeString(Path.of("estudiantes.json"), json);

        // Stream: filtrar y ordenar
        estudiantes.stream()
            .filter(e -> e.promedio() > 9.0)
            .sorted(Comparator.comparingDouble(
                Estudiante::promedio).reversed())
            .forEach(e -> System.out.printf(
                "%s: %.1f\n", e.nombre(), e.promedio()));
    }
}
```

5.3 JavaScript / Node.js

JavaScript ES2024/2025 ha madurado enormemente. Node.js 22 LTS mejora el soporte de módulos ES, rendimiento del motor V8 y las APIs de filesystem. TypeScript se ha convertido en el estándar de facto para proyectos grandes.

```
// Node.js 22 – ESM, fs/promises, top-level await
import { readFile, writeFile } from 'node:fs/promises';

// Clase con validación (estilo moderno)
```

```

class Estudiante {
  #id; #nombre; #promedio;

  constructor(id, nombre, promedio) {
    if (promedio < 0 || promedio > 10)
      throw new RangeError('Promedio fuera de rango');
    this.#id = id;
    this.#nombre = nombre;
    this.#promedio = promedio;
  }

  toJSON() {
    return { id:this.#id, nombre:this.#nombre,
             promedio:this.#promedio };
  }
}

const estudiantes = [
  new Estudiante(1, 'Ana García',    9.5),
  new Estudiante(2, 'Luis Martínez', 8.3),
  new Estudiante(3, 'Sofía Ruiz',    9.8),
];

// Guardar JSON
await writeFile('estudiantes.json',
  JSON.stringify(estudiantes.map(e => e.toJSON()), null, 2));

// Leer, parsear y filtrar
const datos = JSON.parse(
  await readFile('estudiantes.json', 'utf-8'));

datos
  .filter(e => e.promedio > 9.0)
  .sort((a, b) => b.promedio - a.promedio)
  .forEach(e => console.log(`${e.nombre}: ${e.promedio}`));

```

5.4 Go

Go (Golang) fue diseñado por Google para la era de los sistemas distribuidos. Su propuesta de valor: compilación ultra-rápida, goroutines para concurrencia masiva, binarios estáticos sin dependencias, y una sintaxis deliberadamente simple. Es el lenguaje dominante en infraestructura cloud (Docker, Kubernetes, Terraform están escritos en Go).

```

package main

import (
  "encoding/json"
  "fmt"
  "os"
  "sort"
)

```

```

type Estudiante struct {
    ID      int      `json:"id"`
    Nombre  string    `json:"nombre"`
    Promedio float64 `json:"promedio"`
}

func main() {
    estudiantes := []Estudiante{
        {1, "Ana García", 9.5},
        {2, "Luis Martínez", 8.3},
        {3, "Sofía Ruiz", 9.8},
    }

    // Serializar a JSON con sangría
    datos, _ := json.MarshalIndent(estudiantes, "", " ")
    os.WriteFile("estudiantes.json", datos, 0644)

    // Leer y deserializar
    contenido, _ := os.ReadFile("estudiantes.json")
    var cargados []Estudiante
    json.Unmarshal(contenido, &cargados)

    // Ordenar por promedio descendente
    sort.Slice(cargados, func(i, j int) bool {
        return cargados[i].Promedio > cargados[j].Promedio
    })

    for _, e := range cargados {
        if e.Promedio > 9.0 {
            fmt.Printf("%s: %.1f\n", e.Nombre, e.Promedio)
        }
    }
}

```

5.5 Rust

Rust ofrece rendimiento comparable a C/C++ pero con garantías de seguridad de memoria en tiempo de compilación, sin garbage collector. Su sistema de ownership elimina races de datos y dangling pointers. En 2026 Rust está integrado en el kernel de Linux, el navegador Firefox, el motor de Android y muchos sistemas críticos.

```

// Rust 2024 edition – serde para serialización
use std::fs;
use serde::{Serialize, Deserialize};

#[derive(Debug, Clone, Serialize, Deserialize)]
struct Estudiante {
    id:      u32,
    nombre:  String,
    promedio: f64,
}

```



```

}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut estudiantes = vec![
        Estudiante { id:1, nombre:"Ana García".into(), promedio:9.5 },
        Estudiante { id:2, nombre:"Luis Martínez".into(), promedio:8.3 },
        Estudiante { id:3, nombre:"Sofía Ruiz".into(), promedio:9.8 },
    ];

    // Serializar con serde_json
    let json = serde_json::to_string_pretty(&estudiantes)?;
    fs::write("estudiantes.json", &json)?;

    // Leer, deserializar, filtrar y ordenar
    let contenido = fs::read_to_string("estudiantes.json")?;
    let mut cargados: Vec<Estudiante> =
        serde_json::from_str(&contenido)?;

    cargados.sort_by(|a, b|
        b.promedio.partial_cmp(&a.promedio).unwrap());

    for e in cargados.iter().filter(|e| e.promedio > 9.0) {
        println!("{}", e.nombre, e.promedio);
    }
    Ok(())
}

```

5.6 Mojo

Mojo es el lenguaje más reciente en este libro, creado por Modular AI. Diseñado como un superconjunto de Python con características de sistemas de bajo nivel, busca ofrecer la facilidad de Python con el rendimiento de C para aplicaciones de inteligencia artificial y computación científica de alto rendimiento.

```

# Mojo – superconjunto de Python con tipos estáticos opcionales

@value
struct Estudiante:
    var id:      Int
    var nombre:  String
    var promedio: Float64

    fn __init__(inout self, id: Int, nombre: String,
                promedio: Float64):
        self.id      = id
        self.nombre   = nombre
        self.promedio = promedio

    fn mostrar(self):
        print(self.id, self.nombre, self.promedio)

    fn es_destacado(self) -> Bool:

```

```

        return self.promedio > 9.0

fn main():
    let estudiantes = [
        Estudiante(1, "Ana García", 9.5),
        Estudiante(2, "Luis Martínez", 8.3),
        Estudiante(3, "Sofía Ruiz", 9.8),
    ]
    for e in estudiantes:
        if e.es_destacado():
            e.mostrar()

```

5.7 Tabla Comparativa de Lenguajes 2026

Lenguaje	Paradigma principal	Mejor caso de uso	Velocidad	Facilidad
C	Imperativo/Procedural	SO, embebidos, HPC	★★★★★	★★☆☆☆
PHP	Multiparadigma	Web backend, CMS	★★★★☆	★★★★☆
Java	OOP	Enterprise, Android, Big Data	★★★★☆	★★★★☆
JavaScript	Multiparadigma	Web full-stack, Node.js	★★★★☆	★★★★☆
Go	Imperativo+concurr.	Microservicios, infra cloud	★★★★☆	★★★★☆
Rust	Sistemas/funcional	Seguridad crítica, WASM	★★★★★	★★☆☆☆
Mojo	Multiparadigma+AI	IA/ML de alto rendimiento	★★★★★	★★★★☆

Resumen del Capítulo 5

No existe un lenguaje perfecto para todo. C domina en rendimiento puro. Go y Rust lideran en sistemas modernos seguros. Java sigue siendo el estándar enterprise. JavaScript es omnipresente en la web. PHP mantiene su fortaleza en backend web. Mojo emerge como el futuro de la IA de alto rendimiento.

CAPÍTULO 6

Interfaces Gráficas (GUI)

Las interfaces gráficas de usuario transforman aplicaciones de línea de comandos en herramientas accesibles para cualquier usuario, independientemente de su experiencia técnica. El diseño de una buena GUI requiere comprender tanto los aspectos técnicos del framework elegido como los principios de experiencia de usuario.

6.1 GUIs en C con GTK4

GTK4 es el toolkit gráfico estándar en Linux y GNOME. Es una biblioteca C que ofrece widgets modernos, soporte para escalado en pantallas de alta densidad, animaciones y renderizado con aceleración por GPU. Es utilizado por aplicaciones como GIMP, Inkscape y GNOME Files.

```
#include <gtk/gtk.h>

/* Callback: se ejecuta cuando el usuario presiona el botón */
static void on_boton_click(GtkWidget *btn, gpointer datos) {
    GtkLabel *etiqueta = GTK_LABEL(datos);
    gtk_label_set_text(etiqueta, "¡Hola desde GTK4!");
}

/* Se llama cuando la aplicación está lista para mostrarse */
static void activar(GtkApplication *app, gpointer datos) {
    /* Ventana principal */
    GtkWidget *ventana = gtk_application_window_new(app);
    gtk_window_set_title(GTK_WINDOW(ventana), "Mi Primera App");
    gtk_window_set_default_size(GTK_WINDOW(ventana), 400, 250);

    /* Contenedor vertical con espacio entre widgets */
    GtkWidget *caja = gtk_box_new(GTK_ORIENTATION_VERTICAL, 20);
    gtk_widget_set_margin_top(caja, 30);
    gtk_widget_set_margin_bottom(caja, 30);
    gtk_widget_set_margin_start(caja, 40);
    gtk_widget_set_margin_end(caja, 40);

    /* Etiqueta y botón */
    GtkWidget *etiqueta = gtk_label_new("Presiona el botón");
    GtkWidget *boton = gtk_button_new_with_label("Saludar");

    /* Conectar señal 'clicked' con callback */
    g_signal_connect(boton, "clicked",
                     G_CALLBACK(on_boton_click), etiqueta);

    gtk_box_append(GTK_BOX(caja), etiqueta);
    gtk_box_append(GTK_BOX(caja), boton);
    gtk_window_set_child(GTK_WINDOW(ventana), caja);
    gtk_widget_show(ventana);
}
```

```

int main(int argc, char *argv[]) {
    GtkApplication *app = gtk_application_new(
        "com.ejemplo.miapp",
        G_APPLICATION_DEFAULT_FLAGS);
    g_signal_connect(app, "activate",
        G_CALLBACK(activar), NULL);
    int status = g_application_run(
        G_APPLICATION(app), argc, argv);
    g_object_unref(app);
    return status;
}

/* Compilar con:
gcc main.c -o miapp $(pkg-config --cflags --libs gtk4) */

```

6.2 Frameworks GUI por Lenguaje

Lenguaje	Framework	Plataformas	Estilo	Licencia
C/C++	GTK4	Linux, Win, Mac	Nativo GNOME	LGPL
C/C++	Qt 6	Todas + Embedded	QML moderno	GPL/Comercial
C/C++	Dear ImGui	Todas	Inmediata/Dev	MIT
Java	JavaFX	Win, Mac, Linux	CSS styleable	GPL+exception
Python	Tkinter	Todas (incluido)	Clásico	Python License
Python	PyQt6	Todas	Qt moderno	GPL/Comercial
Python	Dear PyGui	Win, Mac, Linux	GPU inmediata	MIT
JavaScript	Electron	Win, Mac, Linux	HTML/CSS/JS	MIT
JavaScript	Tauri	Win, Mac, Linux	HTML/CSS + Rust	MIT/Apache
Rust	Iced	Todas	Elm-inspired	MIT
Go	Fyne	Todas + Mobile	Material-like	BSD 3
Dart	Flutter Desktop	Win, Mac, Linux	Material/Cupertino	BSD 3

6.3 Ejemplo Completo: Tkinter con Python

Tkinter es la opción más accesible para comenzar con GUIs en Python: viene incluida en la instalación estándar, no requiere instalar nada adicional, y tiene una curva de aprendizaje suave. El siguiente ejemplo implementa un gestor de estudiantes completo.

```

import tkinter as tk
from tkinter import ttk, messagebox
import json

class GestorEstudiantes:
    def __init__(self, root):
        self.root = root
        self.root.title('Gestor de Estudiantes - 2026')
        self.root.geometry('700x450')
        self.root.configure(bg='#f0f4f8')
        self.datos = []
        self._construir_ui()

    def _construir_ui(self):
        # — Panel de entrada —
        frame = ttk.LabelFrame(self.root,
                               text=' Agregar Estudiante ', padding=10)
        frame.pack(fill='x', padx=15, pady=10)

        ttk.Label(frame, text='Nombre:').grid(
            row=0, column=0, sticky='w', padx=5)
        self.ent_nombre = ttk.Entry(frame, width=28)
        self.ent_nombre.grid(row=0, column=1, padx=5)

        ttk.Label(frame, text='Promedio:').grid(
            row=0, column=2, sticky='w', padx=5)
        self.ent_prom = ttk.Entry(frame, width=10)
        self.ent_prom.grid(row=0, column=3, padx=5)

        ttk.Button(frame, text='Agregar',
                   command=self.agregar).grid(
            row=0, column=4, padx=10)

        # — Tabla de resultados —
        cols = ('ID', 'Nombre', 'Promedio', 'Estado')
        self.tabla = ttk.Treeview(
            self.root, columns=cols, show='headings', height=12)
        for c in cols:
            self.tabla.heading(c, text=c)
            self.tabla.column(c, width=160)
        self.tabla.pack(fill='both', expand=True, padx=15, pady=5)

        # — Barra de botones —
        bar = ttk.Frame(self.root)
        bar.pack(fill='x', padx=15, pady=8)
        ttk.Button(bar, text='Guardar JSON',
                   command=self.guardar).pack(side='left', padx=5)
        ttk.Button(bar, text='Cargar JSON',
                   command=self.cargar).pack(side='left', padx=5)
        ttk.Button(bar, text='Eliminar',
                   command=self.eliminar).pack(side='right', padx=5)

    def agregar(self):
        nombre = self.ent_nombre.get().strip()
        try:
            prom = float(self.ent_prom.get())
            assert 0 <= prom <= 10

```

```

except (ValueError, AssertionError):
    messagebox.showerror('Error', 'Promedio inválido (0-10)')
    return
if not nombre:
    messagebox.showwarning('Aviso', 'Ingresa un nombre')
    return
nid    = len(self.datos) + 1
estado = 'Aprobado' if prom >= 6.0 else 'Reprobado'
reg    = {'id':nid, 'nombre':nombre,
          'promedio':prom, 'estado':estado}
self.datos.append(reg)
self.tabla.insert('', 'end',
                  values=(nid, nombre, f'{prom:.1f}', estado))
self.ent_nombre.delete(0, 'end')
self.ent_prom.delete(0, 'end')

def guardar(self):
    with open('estudiantes.json', 'w', encoding='utf-8') as f:
        json.dump(self.datos, f, ensure_ascii=False, indent=2)
    messagebox.showinfo('OK', 'Datos guardados en estudiantes.json')

def cargar(self):
    try:
        with open('estudiantes.json', encoding='utf-8') as f:
            self.datos = json.load(f)
        self.tabla.delete(*self.tabla.get_children())
        for r in self.datos:
            self.tabla.insert('', 'end',
                              values=(r['id'], r['nombre'],
                                      f"{r['promedio']:.1f}", r['estado']))
    except FileNotFoundError:
        messagebox.showerror('Error', 'Archivo no encontrado')

def eliminar(self):
    sel = self.tabla.selection()
    if not sel:
        messagebox.showwarning('Aviso', 'Selecciona un registro')
        return
    self.tabla.delete(*sel)

if __name__ == '__main__':
    root = tk.Tk()
    app = GestorEstudiantes(root)
    root.mainloop()

```

Resumen del Capítulo 6

Las GUIs siguen un modelo de eventos: el programa espera acciones del usuario (clics, teclas) y responde a través de callbacks. GTK4 es la opción nativa en C. Qt6 es la más completa para C++. Tkinter es ideal para comenzar en Python. Electron y Tauri permiten usar tecnologías web para construir aplicaciones de escritorio.

CAPÍTULO 7

Práctica Integradora Final

Este capítulo integra todos los conceptos del libro en casos del mundo real. Cada caso está acompañado de análisis de complejidad, decisiones de diseño y explicaciones de las elecciones realizadas. Al final encontrarás ejercicios de completar y corregir código para consolidar el aprendizaje.

7.1 Caso Completo: Sistema de Inventario en C

Implementaremos un sistema de inventario que combina estructuras, archivos binarios, búsqueda, ordenamiento y un menú interactivo. Este es el tipo de programa que se construye en el mundo real para gestionar almacenes, tiendas o bibliotecas.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

typedef struct {
    int    codigo;
    char   descripcion[80];
    char   categoria[30];
    int    cantidad;
    float  precio;
    int    activo;    /* 1=disponible, 0=descontinuado */
} Producto;

#define ARCHIVO  "inventario.dat"
#define MAX_PROD 1000

/* — AGREGAR: O(1) append al final — */
int agregarProducto(const Producto *p) {
    FILE *f = fopen(ARCHIVO, "ab");
    if (!f) { perror("inventario.dat"); return -1; }
    fwrite(p, sizeof(Producto), 1, f);
    fclose(f);
    return 0;
}

/* — BUSCAR por código: O(n) — */
int buscarProducto(int codigo, Producto *res) {
    FILE *f = fopen(ARCHIVO, "rb");
    if (!f) return 0;
    Producto p;
    while (fread(&p, sizeof(Producto), 1, f))
        if (p.codigo == codigo && p.activo) {
            *res = p; fclose(f); return 1;
        }
    fclose(f);
    return 0;
}
```

```

}

/* — LISTAR con estadísticas: O(n) — */
void listarInventario() {
    FILE *f = fopen(ARCHIVO, "rb");
    if (!f) { printf("Sin inventario.\n"); return; }

    Producto p;
    int total = 0;
    float valorTotal = 0.0f;

    printf("\n%-6s %-25s %-15s %6s %10s\n",
           "COD", "DESCRIPCIÓN", "CATEGORÍA", "CANT", "PRECIO");
    printf("%s\n",
           "-----");

    while (fread(&p, sizeof(Producto), 1, f)) {
        if (!p.activo) continue;
        printf("%-6d %-25s %-15s %6d $%.2f\n",
               p.codigo, p.descripcion,
               p.categoria, p.cantidad, p.precio);
        valorTotal += p.cantidad * p.precio;
        total++;
    }
    printf("%s\n",
           "-----");
    printf("Productos activos: %d | Valor total: $%.2f\n",
           total, valorTotal);
    fclose(f);
}

/* — MENÚ interactivo — */
int main() {
    int opcion;
    do {
        printf("\n=== SISTEMA DE INVENTARIO ===\n");
        printf("1. Agregar producto\n");
        printf("2. Buscar producto\n");
        printf("3. Listar inventario\n");
        printf("0. Salir\n");
        printf("Opción: "); scanf("%d", &opcion);

        if (opcion == 1) {
            Producto p = {0};
            printf("Código: "); scanf("%d", &p.codigo);
            printf("Descripción: "); scanf("%79s", p.descripcion);
            printf("Categoría: "); scanf("%29s", p.categoria);
            printf("Cantidad: "); scanf("%d", &p.cantidad);
            printf("Precio: "); scanf("%f", &p.precio);
            p.activo = 1;
            if (agregarProducto(&p) == 0)
                printf("Producto agregado.\n");
        }
        else if (opcion == 2) {
            int cod; printf("Código a buscar: "); scanf("%d", &cod);
            Producto p;
            if (buscarProducto(cod, &p))

```



```

        printf("Encontrado: %s - $%.2f\n",
               p.descripcion, p.precio);
    else
        printf("Producto no encontrado.\n");
    }
    else if (opcion == 3) {
        listarInventario();
    }
} while (opcion != 0);
return 0;
}

```

7.2 Benchmark de Algoritmos de Búsqueda

El siguiente programa mide empíricamente la diferencia de rendimiento entre búsqueda lineal $O(n)$ y búsqueda binaria $O(\log n)$ sobre un millón de elementos.

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define N 1000000

int busquedaLineal(int *arr, int n, int obj) {
    for (int i = 0; i < n; i++)
        if (arr[i] == obj) return i;
    return -1;
}

int busquedaBinaria(int *arr, int n, int obj) {
    int i=0, d=n-1;
    while (i <= d) {
        int m = i + (d-i)/2;
        if (arr[m] == obj) return m;
        else if (arr[m] < obj) i = m+1;
        else d = m-1;
    }
    return -1;
}

int main() {
    int *arr = malloc(N * sizeof(int));
    for (int i = 0; i < N; i++) arr[i] = i; /* Ordenado */

    int objetivo = N - 1; /* Peor caso para búsqueda lineal */
    clock_t t0, t1;

    /* Medir búsqueda lineal */
    t0 = clock();
    for (int r = 0; r < 1000; r++) /* 1000 repeticiones */
        busquedaLineal(arr, N, objetivo);
    t1 = clock();
}

```

```

double ms_lineal = (double)(t1-t0)/CLOCKS_PER_SEC * 1000.0/1000;

/* Medir búsqueda binaria */
t0 = clock();
for (int r = 0; r < 1000; r++)
    busquedaBinaria(arr, N, objetivo);
t1 = clock();
double ms_binaria = (double)(t1-t0)/CLOCKS_PER_SEC * 1000.0/1000;

printf("N = %d elementos\n", N);
printf("Lineal: %.4f ms por búsqueda\n", ms_lineal);
printf("Binaria: %.6f ms por búsqueda\n", ms_binaria);
printf("Ratio de aceleración: %.0fx\n", ms_lineal/ms_binaria);

/* Resultado típico:
   Lineal:  2.1ms   |   Binaria: 0.000020ms   |   ~100.000x más rápida */

free(arr);
return 0;
}

```

7.3 Ejercicios de Completar Código

Ejercicio 1 — Completar: leer archivo y contar palabras

Completa los espacios indicados con `___` para que el programa cuente las palabras de un archivo de texto.

```

#include <stdio.h>
#include <string.h>

int contarPalabras(const char *ruta) {
    FILE *f = fopen(____, ____);    /* 1. abrir para lectura */
    if (____ == NULL) return -1;    /* 2. verificar NULL */

    char palabra[100];
    int contador = 0;

    while (____(palabra, f) == 1)    /* 3. leer palabra por palabra */
        ____;                        /* 4. incrementar contador */

    ____ (f);                        /* 5. cerrar archivo */
    return contador;
}

/* ===== RESPUESTAS =====
1. ruta, "r"
2. f
3. fscanf
4. contador++
5. fclose
===== */

```

Ejercicio 2 — Completar: estructura y función de ordenamiento

```
typedef struct {
    int ____;          /* 1. campo entero para el código */
    char nombre[____]; /* 2. espacio para 60 caracteres */
    float ____;        /* 3. campo decimal para el precio */
} Producto;

/* Ordenar por precio ascendente con Bubble Sort */
void ordenarPrecio(Producto arr[], int n) {
    for (int i=0; i < ____; i++) { /* 4. condición externa */
        for (int j=0; j < ____; j++) { /* 5. condición interna */
            if (arr[j].____ > arr[j+1].____) { /* 6. comparar precios */
                Producto tmp = arr[j];
                arr[j] = ____; /* 7. completar el swap */
                arr[j+1] = tmp;
            }
        }
    }
}

/* ===== RESPUESTAS =====
1. codigo      2. 60      3. precio
4. n-1         5. n-i-1   6. precio, precio
7. arr[j+1]
===== */
```

Ejercicio 3 — Corregir: el código tiene 5 errores

Identifica y corrige los 5 errores del siguiente fragmento:

```
/* —— VERSIÓN CON ERRORES —— */
typedef struct {
    int id
    char nombre[50];
} Persona;

void guardarPersona(Persona p) {
    FILE f = fopen("personas.dat", "wb"); /* ERROR 2: falta * */
    if (f = NULL) return; /* ERROR 3: = vs == */
    fwrite(p, sizeof(Persona), 1, f); /* ERROR 4: p vs &p */
    /* ERROR 5: falta fclose(f) */
}

/* —— VERSIÓN CORREGIDA —— */
typedef struct {
    int id; /* FIX 1: punto y coma */
    char nombre[50];
} Persona;

void guardarPersona(Persona p) {
```

```

FILE *f = fopen("personas.dat", "wb"); /* FIX 2: FILE *f */
if (f == NULL) return;                /* FIX 3: == */
fwrite(&p, sizeof(Persona), 1, f);     /* FIX 4: &p */
fclose(f);                             /* FIX 5: cerrar */
}

```

7.4 Proyecto Final Integrador

El proyecto final integra los 7 capítulos del libro en una aplicación completa. Se recomienda implementarlo en fases, verificando cada una antes de continuar.

Enunciado del Proyecto: Sistema de Biblioteca

Implementa un sistema completo de gestión de biblioteca que permita: registrar libros (ISBN, título, autor, año, copias disponibles), registrar préstamos (qué usuario llevó qué libro y cuándo), devolver libros y calcular multas por retraso, generar reportes de libros más prestados, y buscar por ISBN (búsqueda binaria) y por autor (búsqueda lineal).

Especificaciones técnicas del proyecto:

1. Definir struct Libro con campos: isbn[20], titulo[100], autor[60], anio, copias_total, copias_disp.
2. Definir struct Prestamo con: id_prestamo, isbn[20], id_usuario, fecha_prestamo[11], fecha_devolucion[11], devuelto.
3. Guardar libros en "biblioteca.dat" (binario) y préstamos en "prestamos.dat" (binario).
4. Implementar búsqueda por ISBN con búsqueda binaria ($O(\log n)$) sobre un array cargado en memoria.
5. Implementar búsqueda por autor con búsqueda lineal ($O(n)$).
6. Ordenar libros por título con Merge Sort ($O(n \log n)$) al generar reportes.
7. Registrar logs de operaciones en "logs.txt" (archivo de texto, modo append).
8. (Opcional avanzado) Interfaz gráfica con GTK4 o Python Tkinter.

Criterio de evaluación	Descripción	Peso
Correctitud	Compila sin warnings, maneja errores de archivos	30%
Complejidades	Usa Big-O óptimos para cada operación	25%
Modularidad	Funciones pequeñas, máximo 40 líneas cada una	20%
Documentación	Cada función tiene comentario con complejidad	15%
Robustez	Maneja archivos inexistentes y entradas inválidas	10%

Resumen del Capítulo 7

Los proyectos integradores son la mejor forma de consolidar el aprendizaje. La combinación de struct + archivos binarios + algoritmos de búsqueda y ordenamiento es el núcleo de los sistemas de gestión de datos en C. Los ejercicios de completar y corregir código desarrollan la habilidad de leer y depurar código ajeno, esencial en el mundo profesional.

BIBLIOGRAFÍA

Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional.

Klabnik, S., & Nichols, C. (2023). The Rust Programming Language (2nd ed.). No Starch Press.

Donovan, A. A. A., & Kernighan, B. W. (2015). The Go Programming Language. Addison-Wesley Professional.

Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley Professional.

Mozilla Developer Network. (2024). JavaScript Reference. <https://developer.mozilla.org/>

GTK Development Team. (2024). GTK 4 Reference Manual. <https://docs.gtk.org/gtk4/>

Python Software Foundation. (2024). Python 3 Documentation — tkinter. <https://docs.python.org/3/library/tkinter.html>

Modular AI. (2024). Mojo Programming Manual. <https://docs.modular.com/mojo/>

— *Fin de la obra* —

Yuber Anthony Ticona Incacutipa · Lenguajes de Programación I · Edición 2026